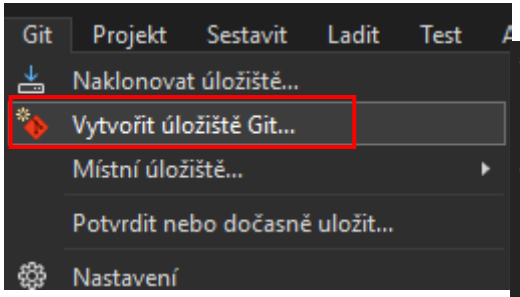
Propojení C# projektu GitHubem

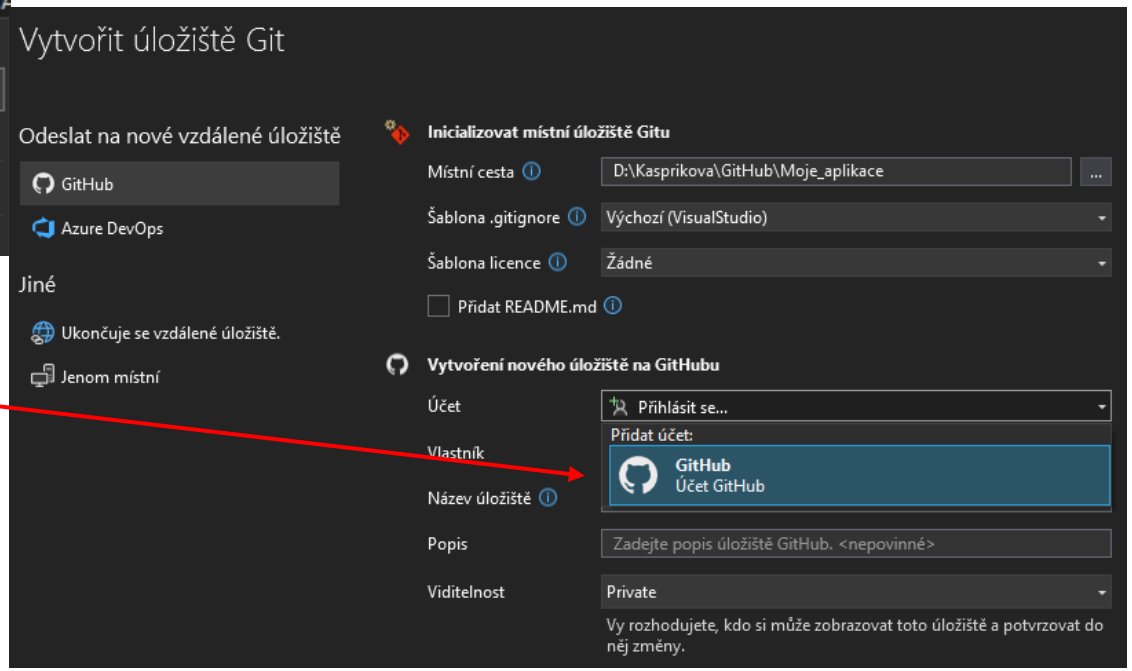
Ing. Eva Kaspříková

Přidání projektu do GitHubu

- Vytvořte projekt (např. Console App)
- Zapněte Git:
  - Git → Vytvořit úložiště Git

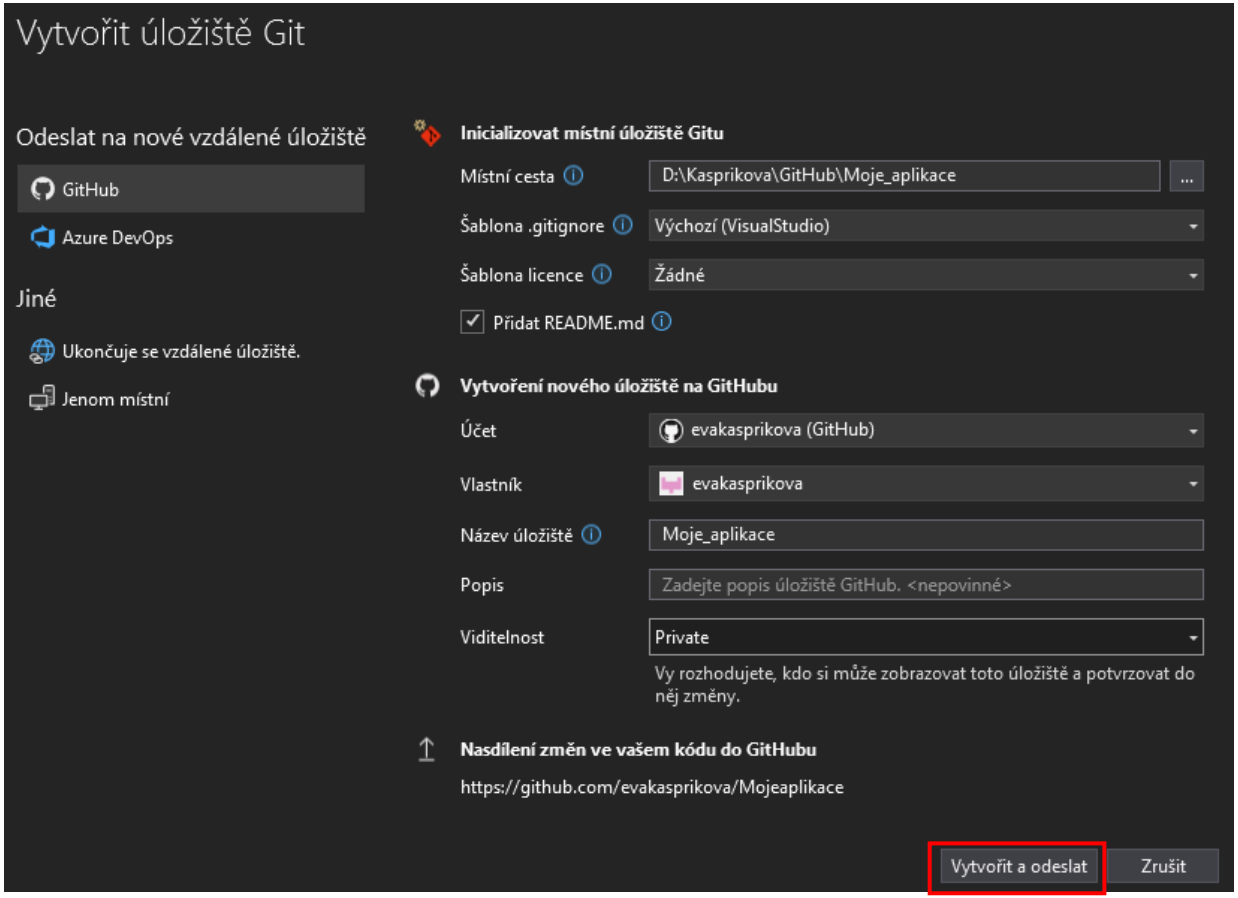


- přihlaste se

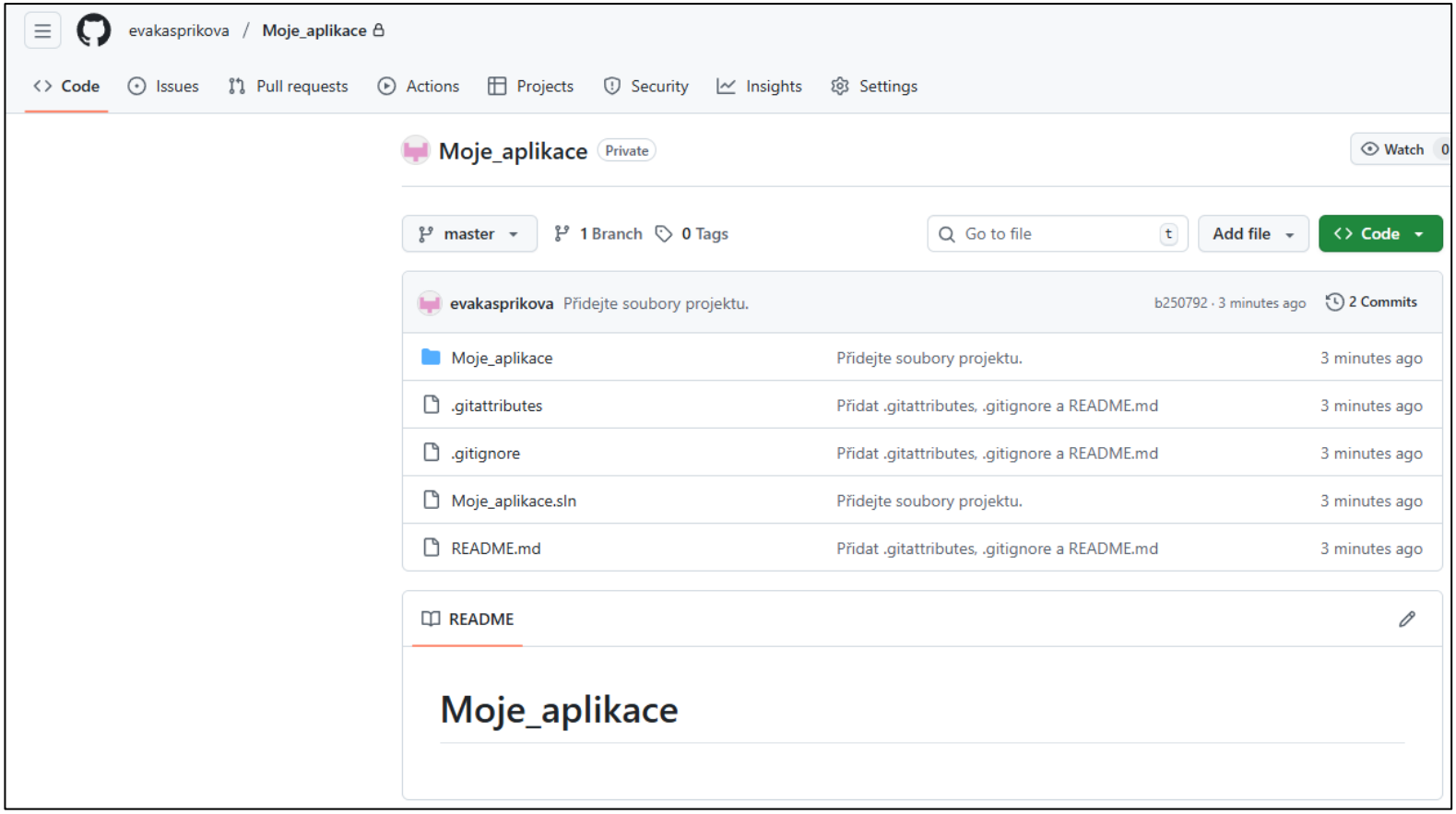


Nastavení repozitáře

- Nastavte repozitář a vytvořte

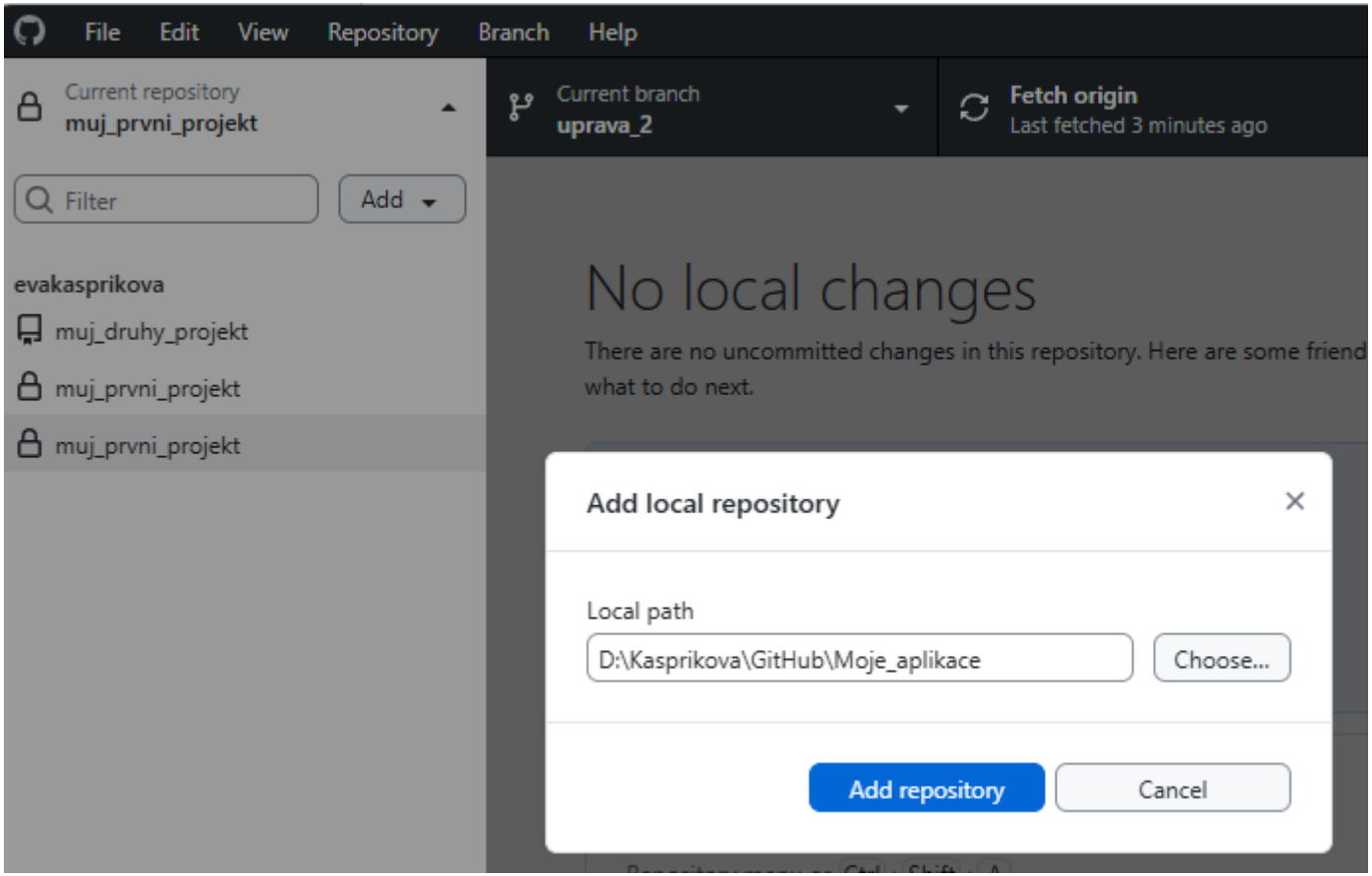


Vytvořený repozitář na GitHubu

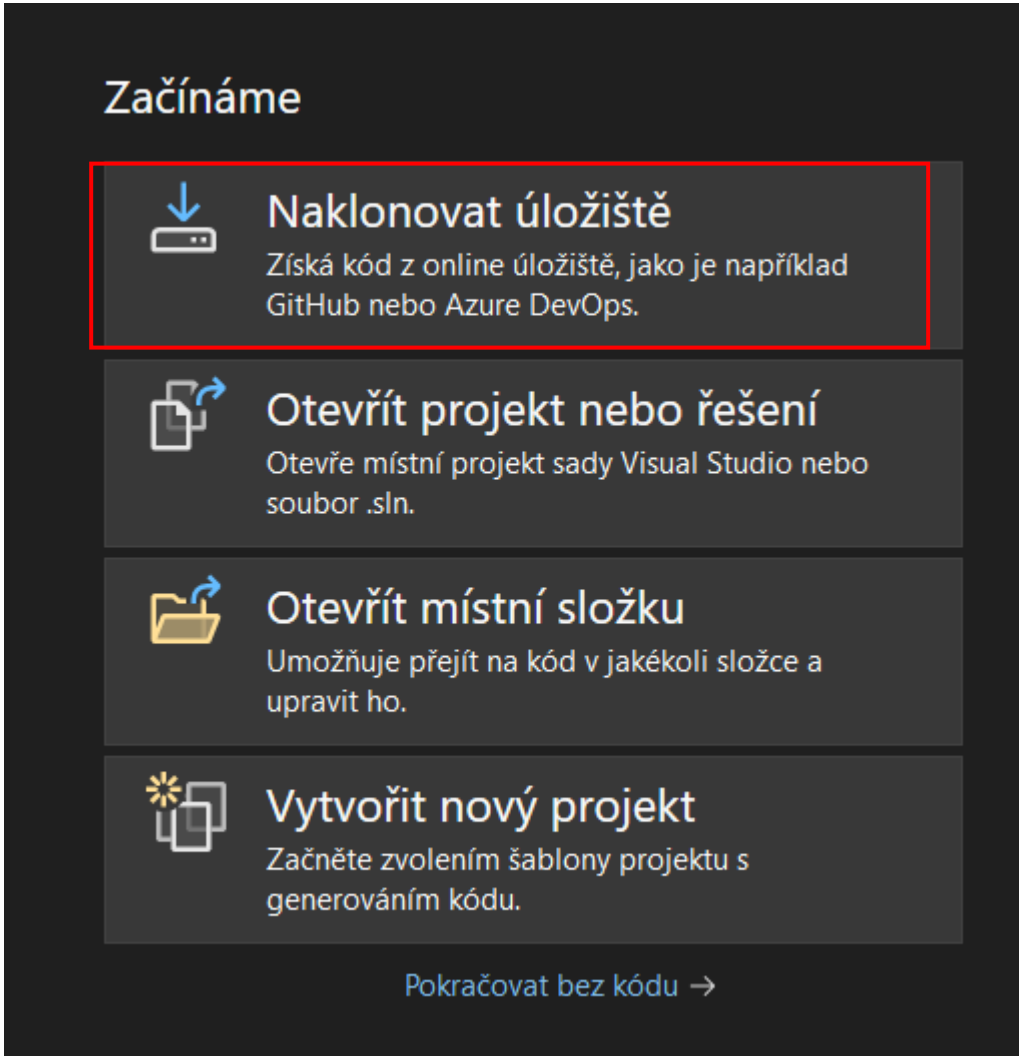


Přidání lokálního repozitáře do GitHub Desktopu

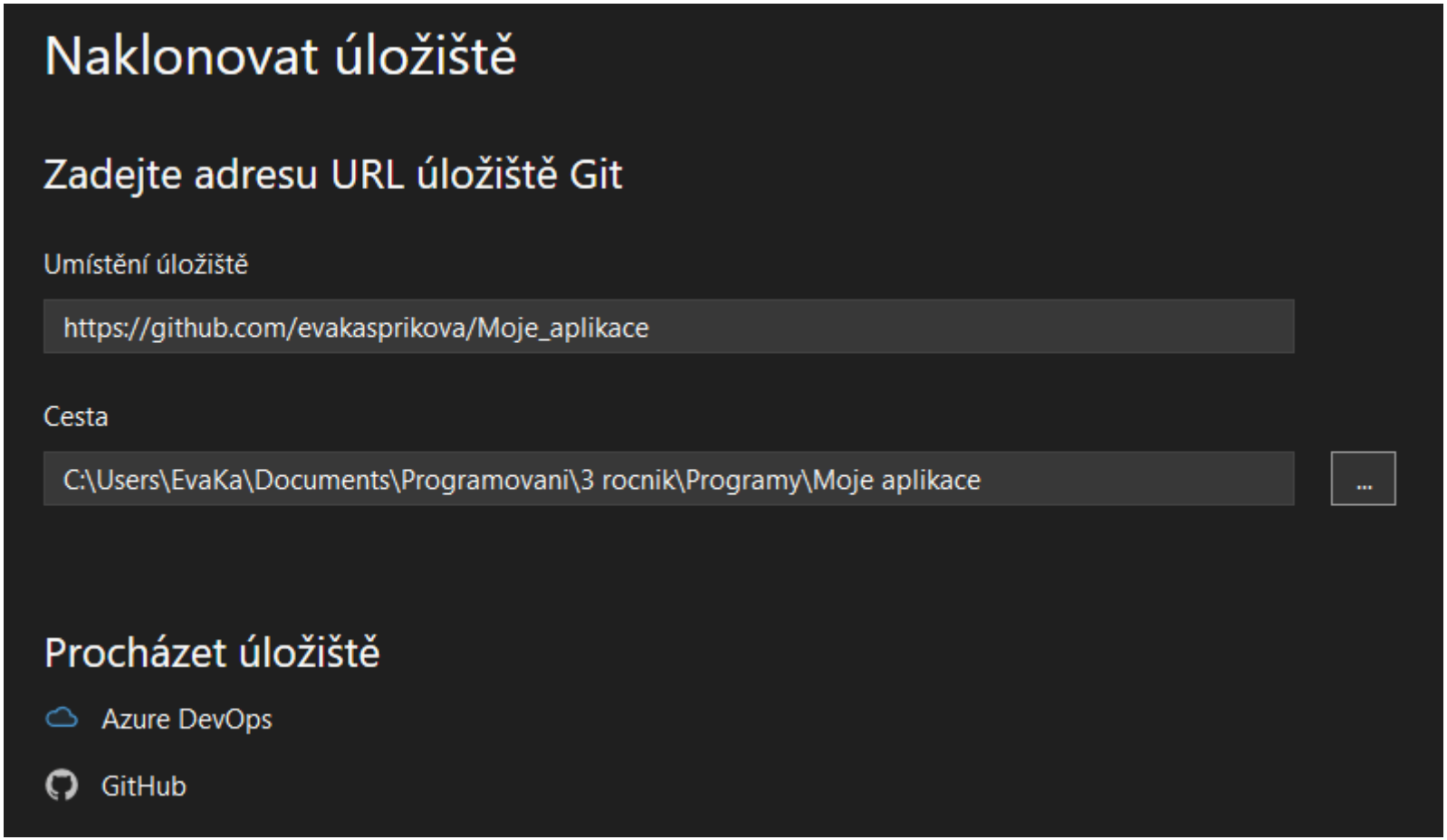
- Add > Add existing repository



Naklonování úložiště z GitHubu

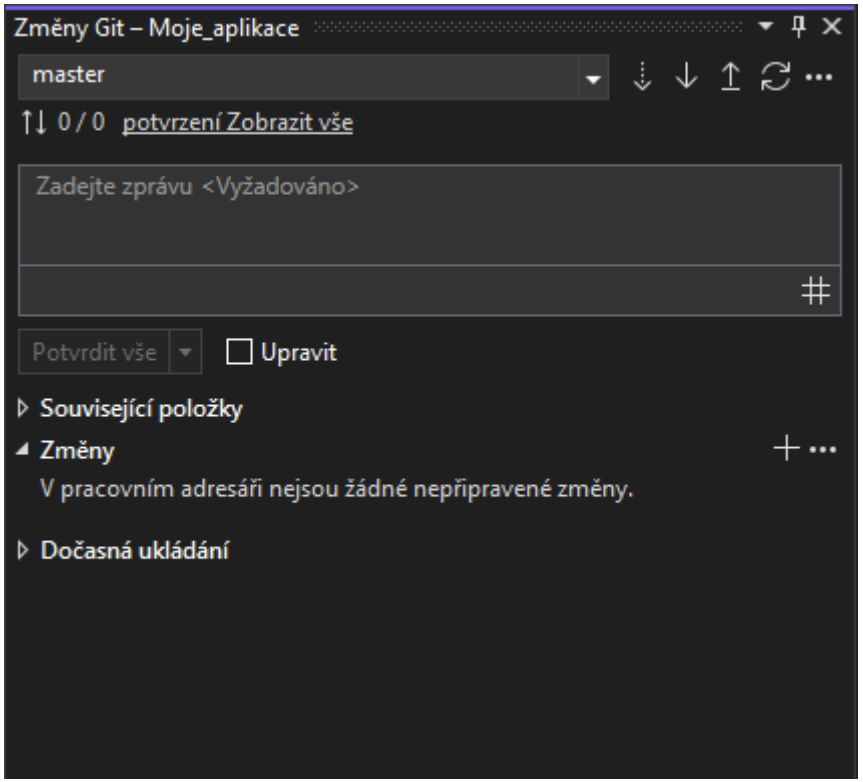


Naklonování úložiště z GitHubu



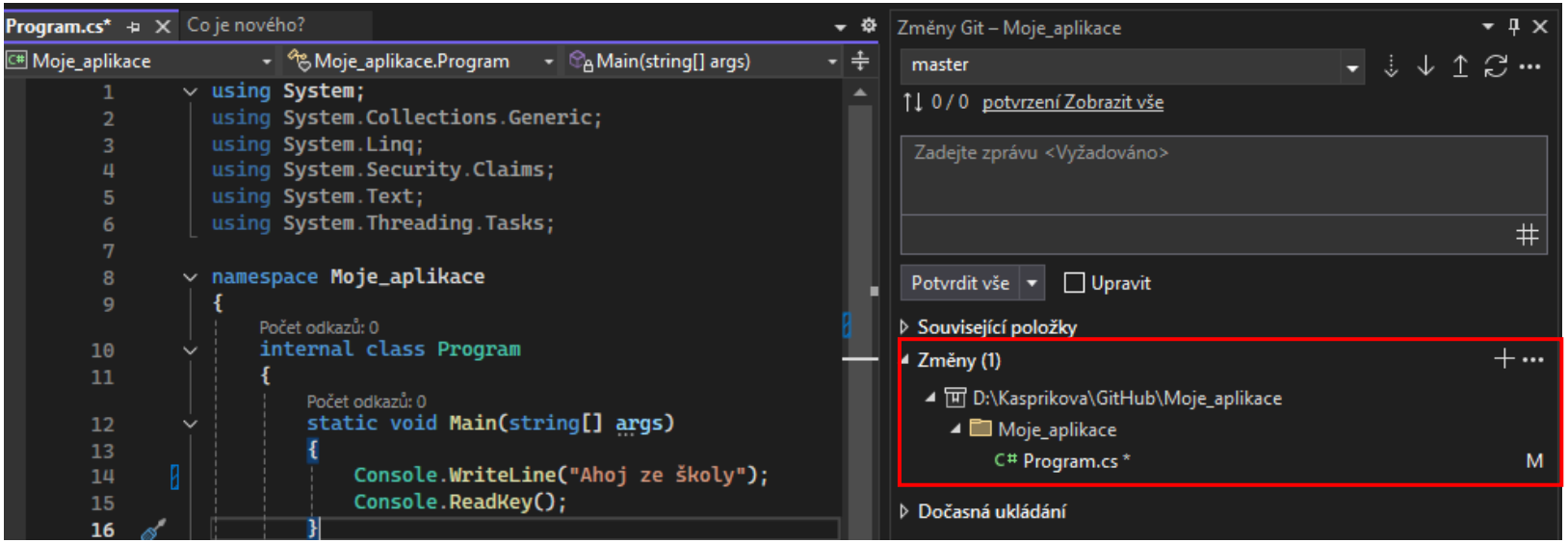
Okno Změny Git (Git Changes)

- Zobrazit > Změny Git (Ctrl + Alt + F7)



Změna v kódu

- Git rozpoznal změnu



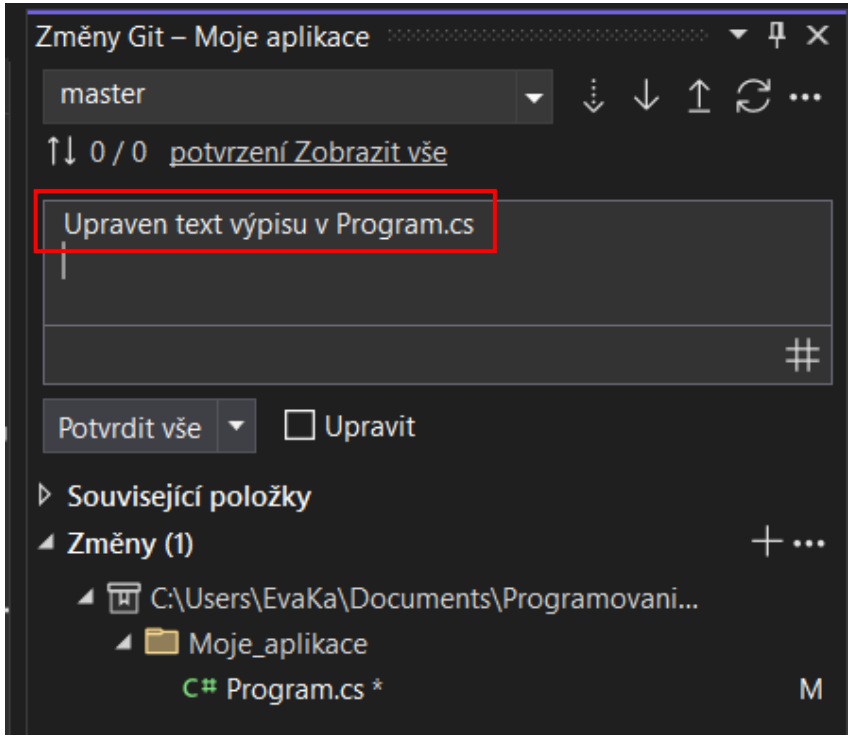
Popis změny

- Vpravo nahoře je pole

  „Zadejte zprávu <Vyžadováno>“.

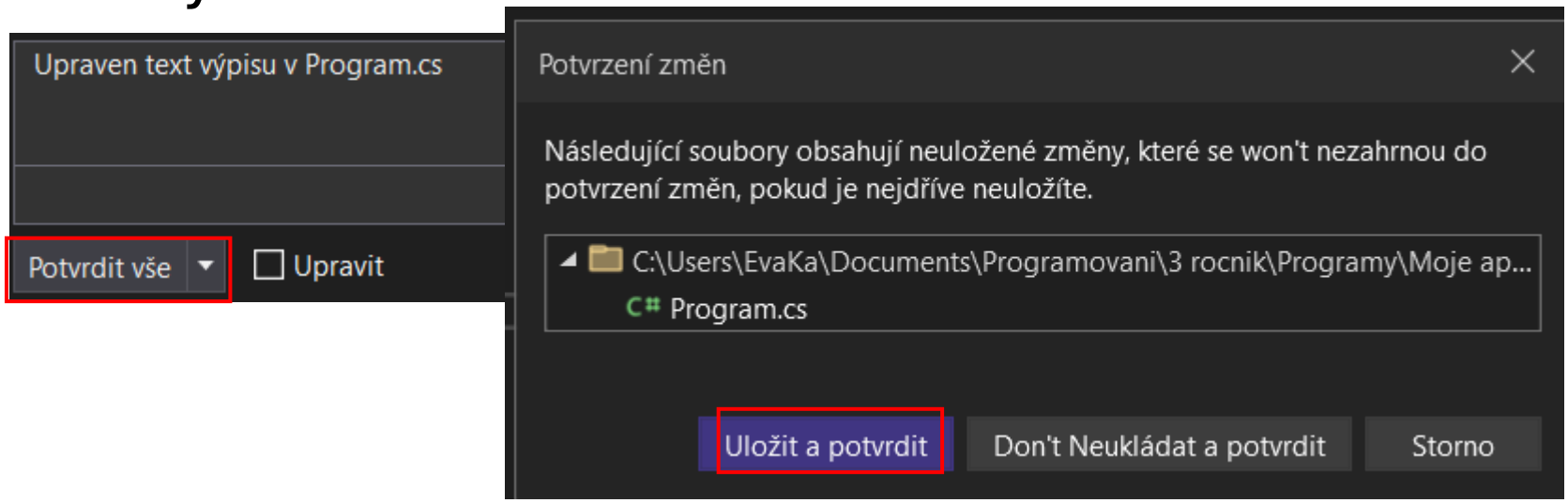
- Napište stručně, co jste upravili – např.:

  Upraven text výpisu v Program.cs



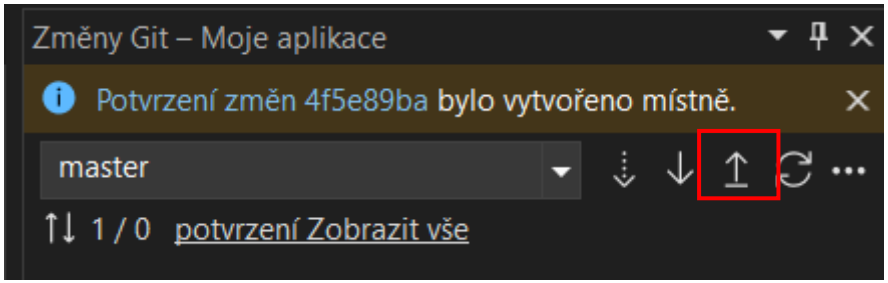
Vytvoření commitu

- Klikněte na *Potvrdit vše*
- (tím se vytvoří **commit**)
- V seznamu změn (vpravo) zmizí hvězdička a Visual Studio napíše, že projekt nemá neuložené změny.

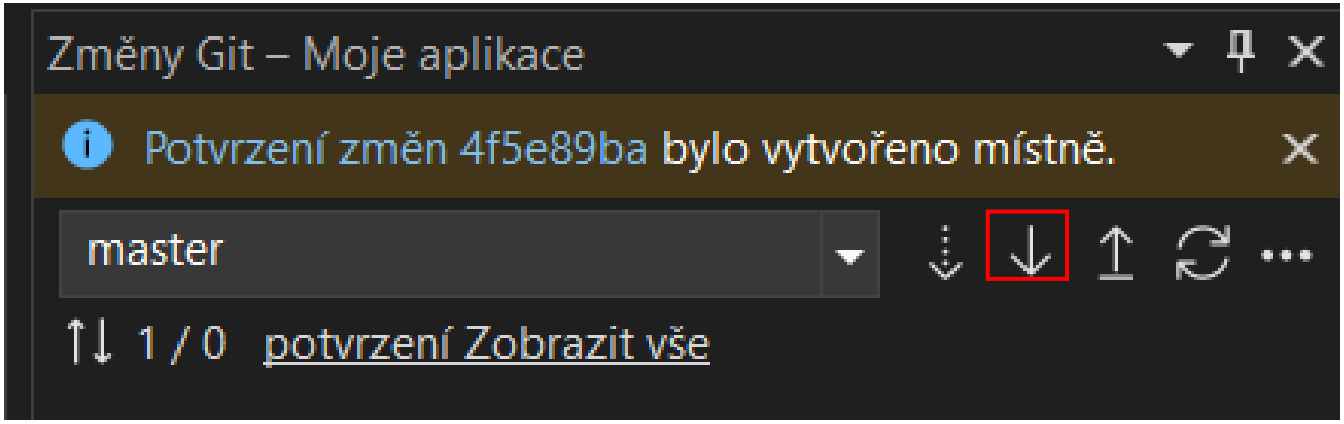


Odeslání na GitHub (push)

- Klikněte na malou šipku nahoru (Push) – nebo nahoře v liště Git → Push.

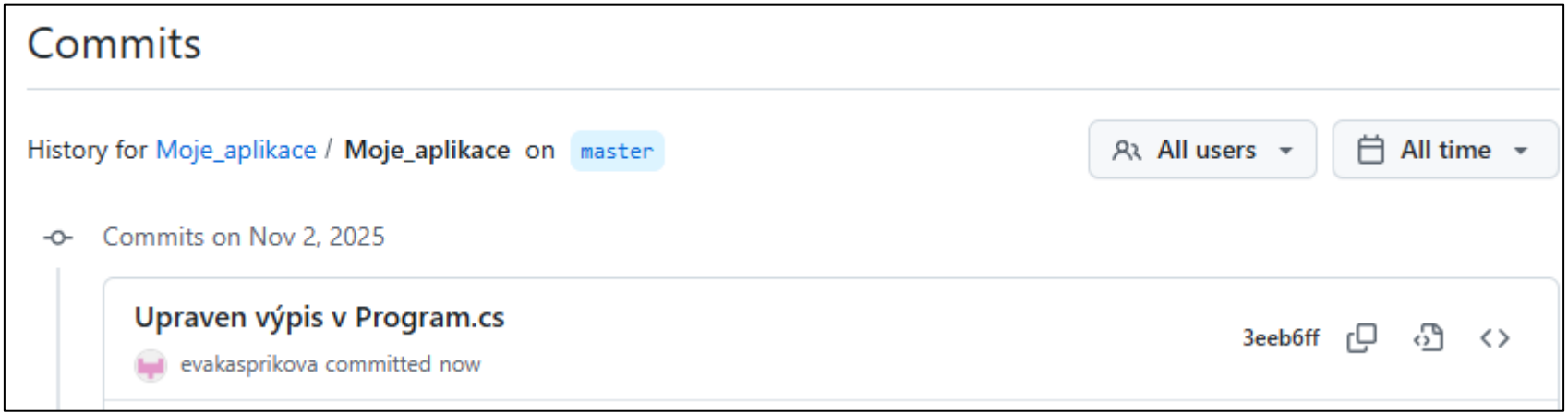


Stažení změn z GitHubu



Ověření na GitHubu

- Otevřete si svůj repozitář na GitHub.com.
- Otevřete soubor Program.cs.
- Klikněte na záložku **History** nebo **Commits**



Přidání nového souboru

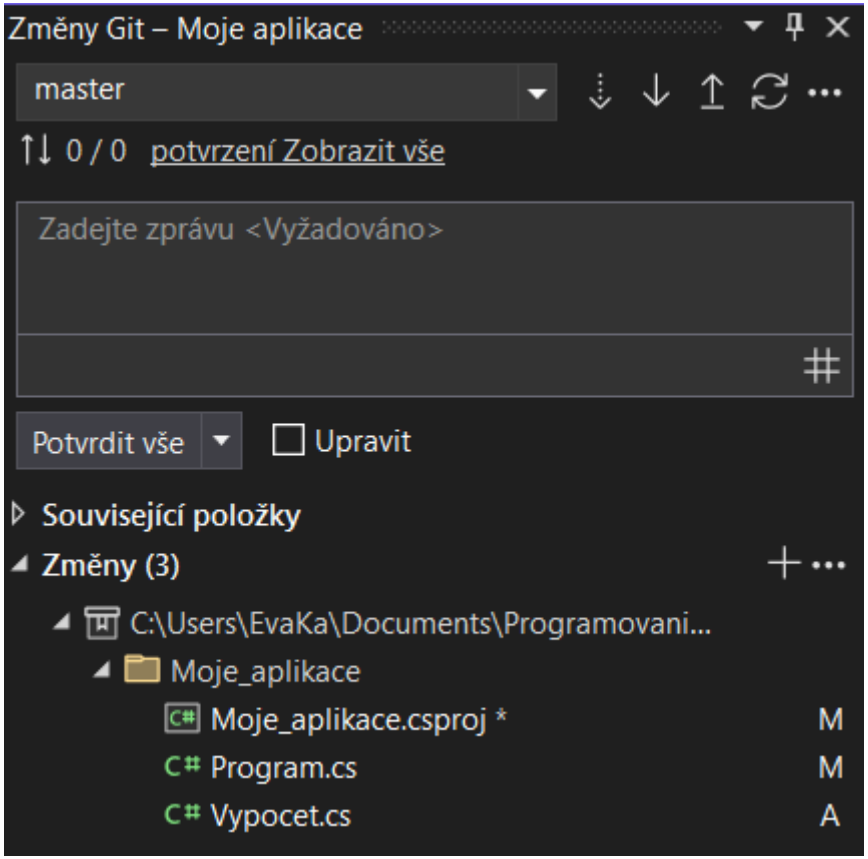
- Klikněte pravým tl. na složku projektu
- Zvolte **Přidat → Třída Vypocet.cs**
- Přidejte kód a uložte

```
namespace Moje_aplikace
{
    Počet odkazů: 0
    internal class Vypocet
    {
        Počet odkazů: 0
        public static int Secti(int a, int b)
        {
            return a + b;
        }
    }
}
```

Do program.cs napište

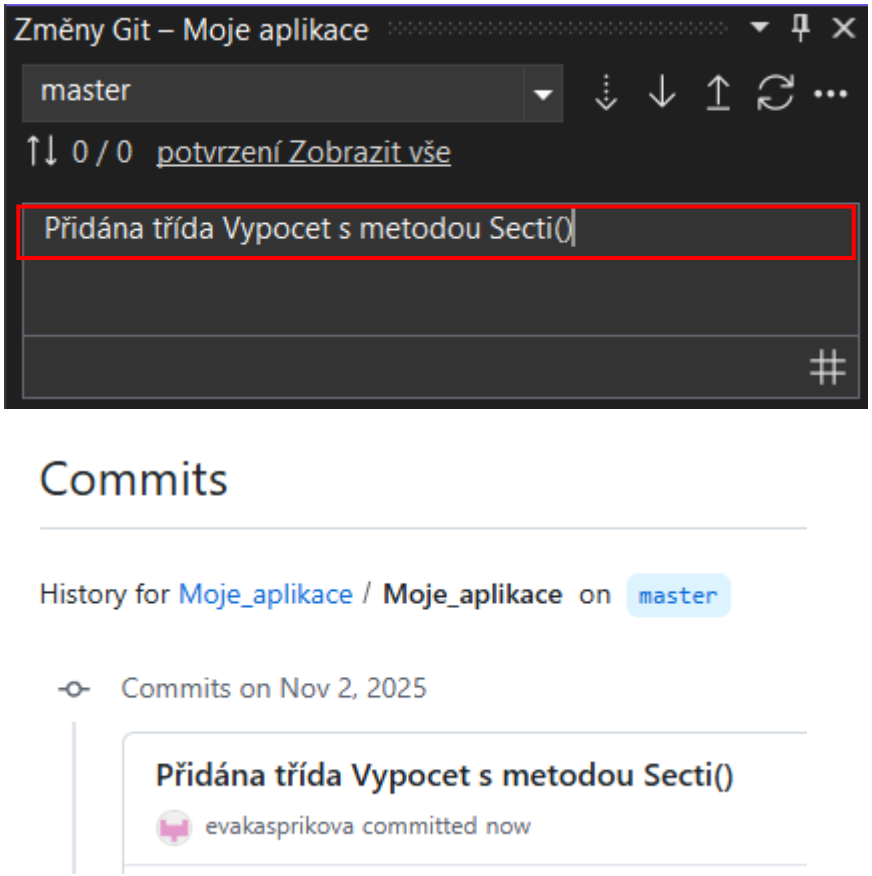
```
static void Main(string[] args)
{
    Console.WriteLine("Ahoj ze školy");
    Console.WriteLine(Vypocet.Secti(5, 7));
    Console.ReadKey();
}
```

Změny Git


Změny Git – Moje aplikace
master
0 / 0
potvrzení Zobrazit vše
Zadejte zprávu <Vyžadováno>
Potvrdit vše
Upravit
Související položky
Změny (3)
C:\Users\EvaKa\Documents\Programovani...
Moje_aplikace
Moje_aplikace.csproj *
M
Program.cs
M
Vypocet.cs
A

Commit

- Napište zprávu, potvrďte a pushněte



Historie změn

evakasprikova Přidána třída Vypocet s metodou Secti()

Code | **Blame** | 19 lines (18 loc) · 409 Bytes

Older Newer

4 days ago	Přidejte soubory projektu.	1	`using System;`
		2	`using System.Collections.Generic;`
		3	`using System.Linq;`
		4	`using System.Security.Claims;`
		5	`using System.Text;`
		6	`using System.Threading.Tasks;`
		7	
		8	`namespace Moje_aplikace`
		9	`{`
		10	`    internal class Program`
		11	`    {`
		12	`        static void Main(string[] args)`
		13	`        {`
1 hour ago	Upraven výpis v Program.cs	14	`            Console.WriteLine("Ahoj ze školy");`
4 minutes ago	Přidána třída Vypocet s metodo...	15	`            Console.WriteLine(Vypocet.Secti(5, 7));`
4 days ago	Přidejte soubory projektu.	16	`            Console.ReadKey();`
		17	`        }`
		18	`    }`
		19	`}`

Jak číst změny kódu

- Každý blok kódu má:
  - autora změny
  - čas, kdy byla změna provedena,
  - zprávu commitu (např. „Upraven výpis v Program.cs“),
  - přímo vedle je kód, který patří k tomuto commitu.

- Git neukládá celý projekt pokaždé znovu
- Sleduje každou změnu po řádcích – kdo, kdy a co upravil.

Soubor `.gitignore`

- říká Gitu, které soubory nebo složky má ignorovat – tedy *nevšímat si jich a neukládat je do verzovací historie*
- V projektu vzniká spousta souborů, které:
  - se automaticky generují při kompilaci (např. bin/, obj/),
  - obsahují osobní nastavení vývojového prostředí (např. .vs/, .user, .suo),
  - nemají být veřejné (např. hesla, API klíče, lokální konfigurace).

Typické položky v .gitignore pro C#

```
# Složky s přeloženými soubory
bin/
obj/

# Nastavení Visual Studia
.vs/
*.user
*.suo

# Výsledky testů
TestResults/

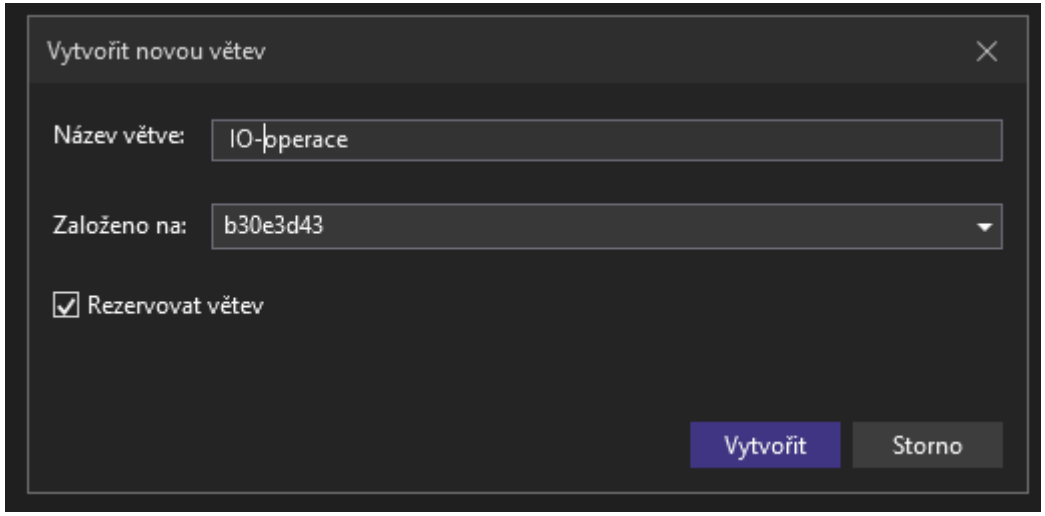
# Zálohy nebo dočasné soubory
*.tmp
*.log
```

Příklad

- Otevři svůj projekt ve Visual Studiu.
- V kořenové složce vytvoř soubor .gitignore.
- Zapiš do něj:
  - bin/
  - obj/
  - *.log
- Vytvoř v projektu složku bin a do ní soubor test.log.
- Podívej se do **Změn Git** — soubor test.log tam **není vidět**, protože Git ho ignoruje.

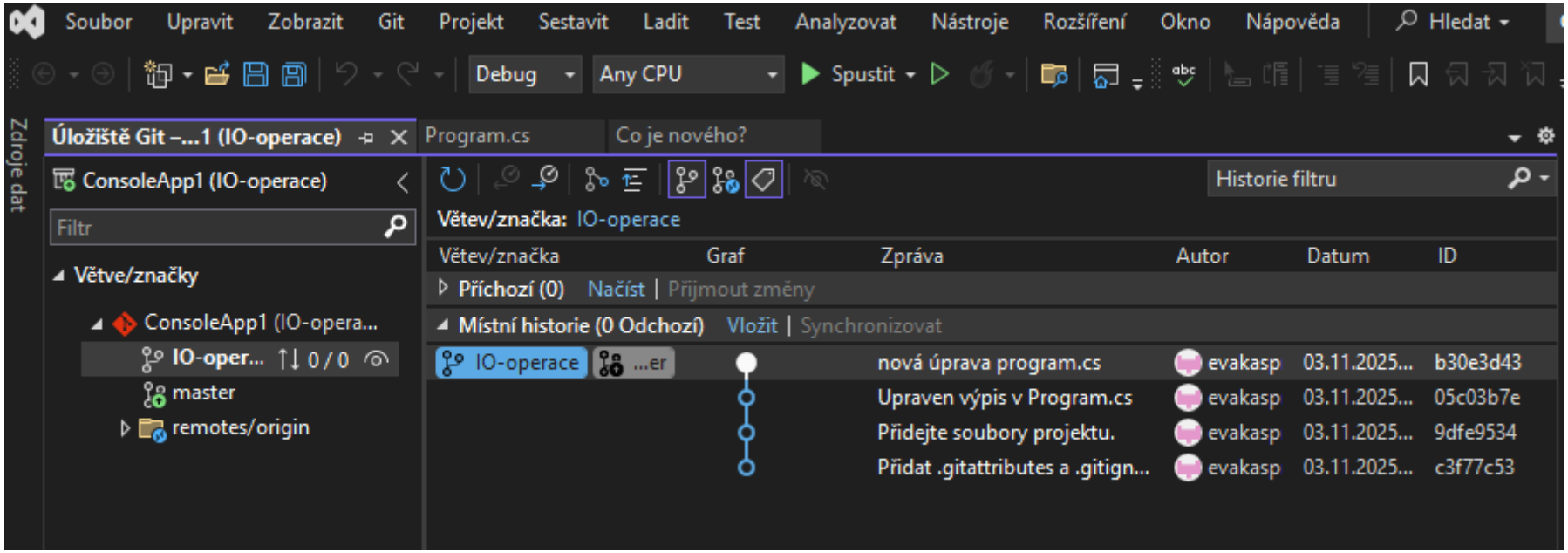
Vytvoření větve

- Git > Nová větev



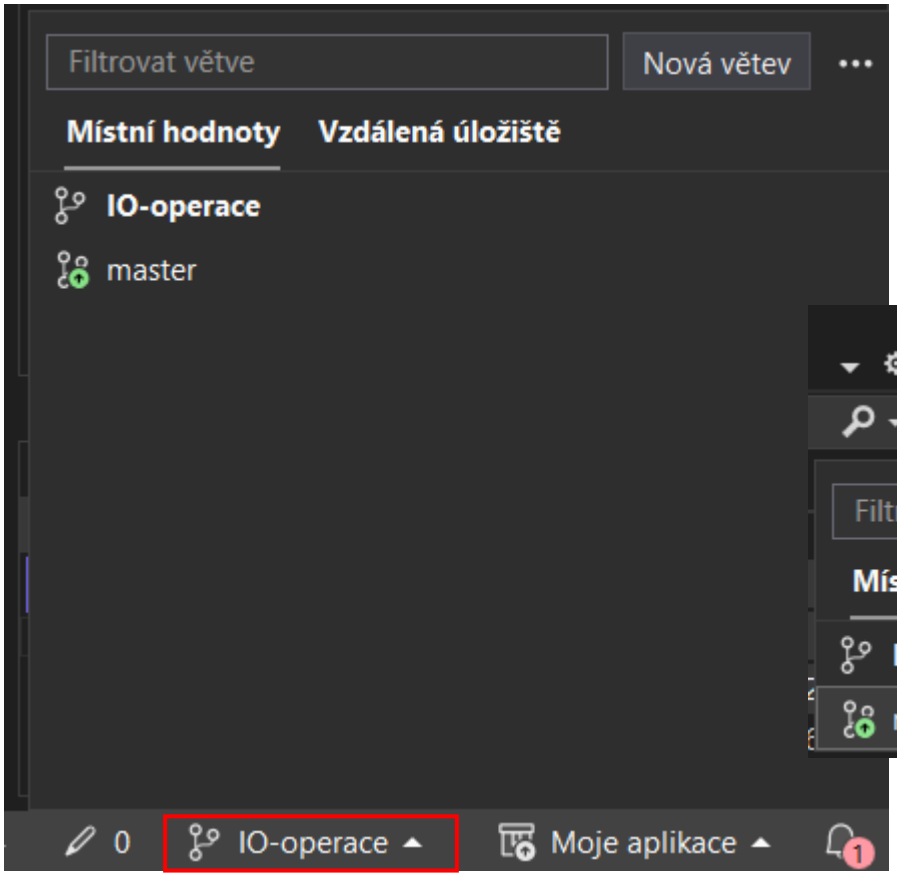
Správa větví

- Git > Spravovat větve

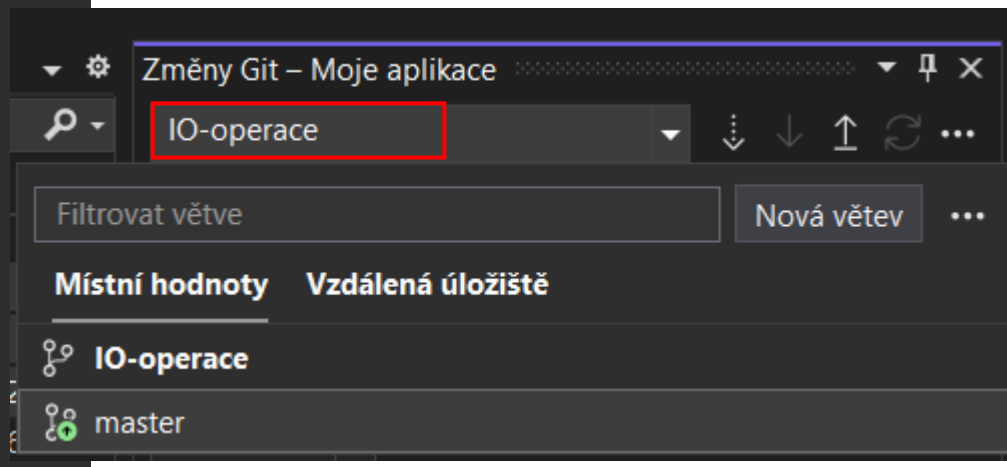


Přepínání větví

- V pravé dolní části okna rozklikneme a vybereme

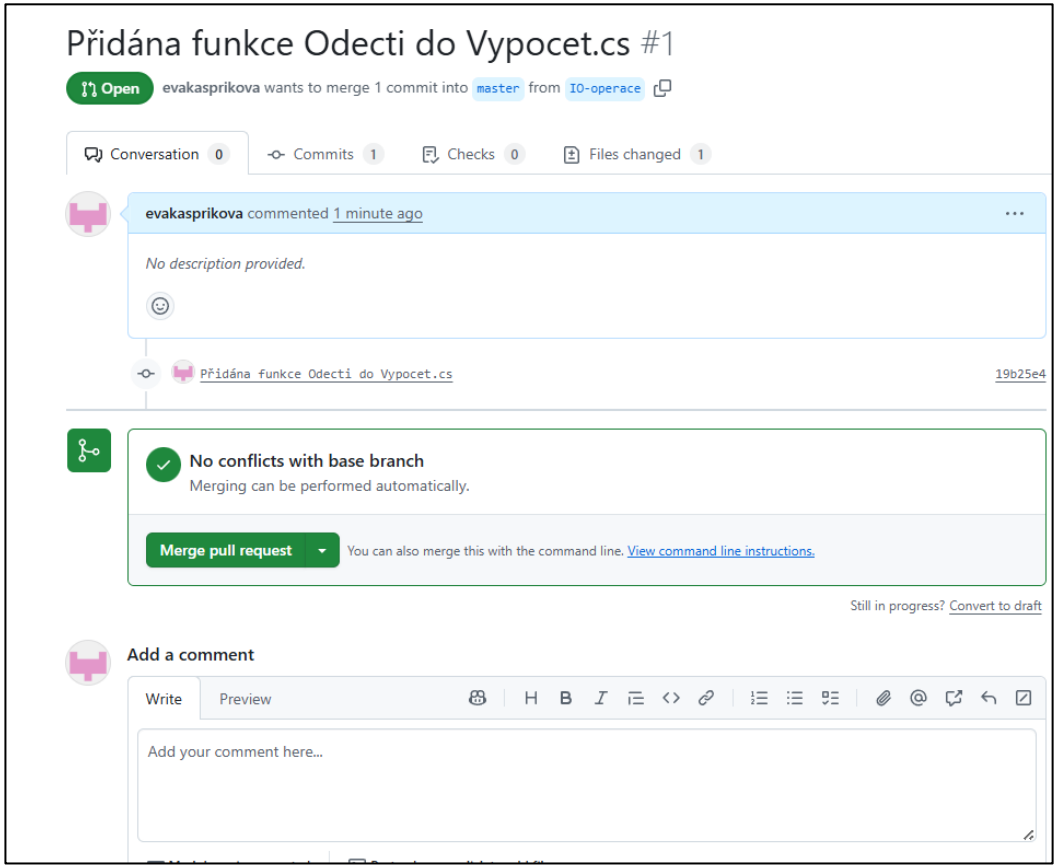


V okně Změny Git



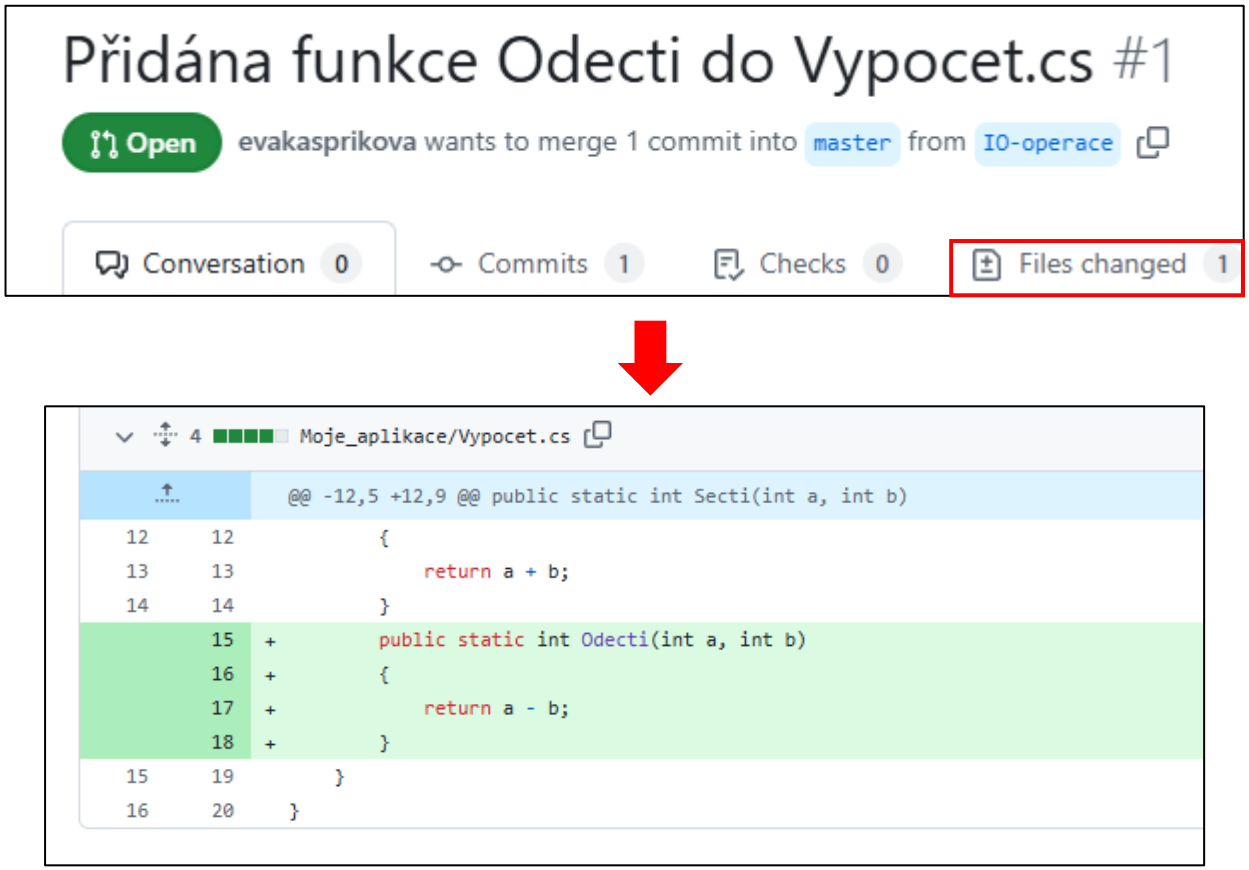
Pull Request

- Commit změny
- Push
- Pull Request



Kontrola změn

- Klikněte na Files changed



Týmový projekt: Útulek pro zvířata

- Vytvořte konzolovou aplikaci v C#, která eviduje zvířata v útulku, umožní s nimi základní práci a uloží data do souboru. Tým pracuje ve větvích, změny slučuje přes Pull Requesty.

Funkční požadavky

1. Přidání zvířete (pes/kočka/„jiné“), vlastnosti:
   - ID (automaticky), jméno, druh, věk (roky), pohlaví, datum příjmu, zdravotní stav, poznámka
2. Výpis všech zvířat (přehledná tabulka)
3. Vyhledávání/filtr:
   - podle druhu, věku (≤/≥), jména
4. Označení adopce (adoptováno: ano/ne, datum adopce)
5. Základní validace vstupů (věk je číslo ≥ 0, nepovinné kolonky povolit prázdné)

pozn. Zvířata ukládejte do Listu (můžete v kódu přímo načíst nějaká zvířata, abyste je po každém spuštění nemuseli zadávat ručně), lze použít zápis a načtení ze souboru (volitelné).

Rozšíření

- Statistiky
  - počet zvířat podle druhu
  - průměrný věk
  - počet adoptovaných
- Editace a mazání zvířete
- Export „karta zvířete“ do TXT
- Načítání a ukládání do souboru

Struktura řešení

```
Utulek/
│
├── Program.cs        // hlavní spouštěcí třída + menu aplikace
│
├── Model/
│   └── Zvire.cs      // třída modelu zvířete (vlastnosti: ID, jméno, druh, věk, pohlaví, adoptováno)
│
├── Services/
│   └── Evidence.cs   // logika CRUD operací nad List<Zvire> (přidání, úprava, mazání, výpis, adopce)
│
└── UI/
    └── KonzoleUI.cs  // komunikace s uživatelem – čtení vstupů a zobrazení výpisů
```

Uživatelské menu (konzola)

===== ÚTULEK PRO ZVÍŘATA =====

1) Přidat zvíře

2) Vypsat všechna zvířata

3) Vyhledat/ filtrovat

4) Označit adopci

5) Statistiky (volitelné)

0) Konec

Volba: _

Tým a větve (doporučení)

- 3-4 členové týmu, jeden vedoucí
- Každý člen týmu má vlastní větev, vek teré pracuje na své části kódu
- Každý člen udělá minimálně 3 commity

- Commity a Pull Requesty pojmenovávejte smysluplně např. „Přidána evidence + filtry“

Git workflow

- Vlastní větev → práce → commit (výstižný message).
- Push na GitHub.
- Pull Request → code review (vedoucí).
- Merge do main po schválení.
- Tým pravidelně dělá Pull main.

Odevzdání

- Odkaz na repozitář (public) + krátký README.md:
  - popis funkcí,
  - rozdělení rolí,
  - jak spustit
- Každý student: odkaz na svůj Pull Request + 1 odstavec reflexe.